

Django 教學 13

訂單結帳與交易

第 19 章

從最小範例到專案實作與驗收

本份學習路線

先完成 [12_購物車與流程測試](#)。

- **第 19 章：訂單與交易一致性**

每章依序：概念、最小範例、語法、專案對照、實作與驗收。

[全課目錄](#) · [實作手冊對照](#)

第 19 章

訂單與交易一致性

驗證結帳快照、庫存與購物車變化，讓中途失敗證明 rollback。

本章的操作環境與成果

在 LearnMart 的練習分支操作；LearnBoard 用來比較相同規則。

完成成果： 驗證結帳快照、庫存與購物車變化，讓中途失敗證明 rollback。

完整範例可依步驟操作；標示「節錄／重排」的程式用來閱讀，不當作整檔覆蓋。
進階頁可回查，但所有基本驗收需完成。

19-1 為什麼需要 Order 與 OrderItem ?

Order (訂單主檔)

- buyer、收件資料、狀態、總額、時間

- OrderItem (每一項商品)

 - product、seller

 - product_name、unit_price 快照

 - quantity

一張訂單可包含多項商品；每項商品又需要購買當下的資料。

19-2 Order 保存交易層資訊

```
class Order(models.Model):
    buyer = models.ForeignKey(User, on_delete=models.PROTECT,
                              related_name="orders")
    recipient_name = models.CharField(max_length=80)
    address = models.CharField(max_length=255)
    phone = models.CharField(max_length=30)
    status = models.CharField(...)
    total = models.DecimalField(...)
```

PROTECT 避免刪除 buyer 時破壞歷史訂單關聯。

19-3 OrderItem 為什麼同時有 FK 與快照？

```
class OrderItem(models.Model):  
    product = models.ForeignKey(Product, on_delete=models.PROTECT)  
    seller = models.ForeignKey(User, on_delete=models.PROTECT)  
    product_name = models.CharField(max_length=150)  
    unit_price = models.DecimalField(...)  
    quantity = models.PositiveIntegerField()
```

- FK 保留追蹤關係
- `product_name` / `unit_price` 保留購買時歷史
- 商品日後改名或漲價，舊訂單不應跟著改

19-4 訂單總額也由伺服器保存快照

危險：

```
<input type="hidden" name="total" value="1980">
```

hidden input 仍可修改。正確：

```
order.total = sum(item.subtotal for item in items)
```

價格、seller、buyer、status 都是 server-owned data，不列入 CheckoutForm。

19-5 CheckoutForm 的 allowlist

目前 LearnMart | 節錄／重排 | [marketplace/forms.py](#)

```
class CheckoutForm(BootstrapFormMixin, forms.ModelForm):  
    class Meta:  
        model = Order  
        fields = ("recipient_name", "phone", "address")
```

使用者只輸入收件資料。Order 的 buyer、total、status 與 items 由 View 根據 authenticated user 與 database state 建立。

19-6 Checkout 的入口先處理身份與 cart

```
@login_required
@transaction.atomic
def checkout(request):
    items = list(
        request.user.cart_items
        .select_related("product", "product__seller")
        .select_for_update()
    )
    if not items:
        messages.info(request, "購物車目前是空的。")
        return redirect("marketplace:cart")
```

目前 implementation 在 GET 與 POST 都先 materialize queryset；後面會討論鎖定位置。

19-7 list(queryset) 讓查詢立即執行

QuerySet 原本 lazy；轉成 list 會立刻讀資料。

這裡的效果：

- 後續多次迭代使用同一批 CartItem object
- 若 backend 支援 `select_for_update()`，evaluation 時才嘗試取得鎖
- 鎖定必須在 transaction 內才有意義

19-8 GET 與 POST 在 checkout 扮演不同角色

- GET：建立 unbound CheckoutForm，顯示收件 form 與 cart summary，不應改資料
- POST：建立 bound form，驗證收件資料後建立訂單、扣庫存、清 cart

目前 `@transaction.atomic` 包住兩者。教學上要看懂現況，也要知道更精準的設計可只在 POST mutation 階段進 transaction / lock。

19-9 POST 先建立 bound CheckoutForm

```
form = CheckoutForm(request.POST or None)
if request.method == "POST" and form.is_valid():
    ...
```

這是目前原始碼的簡寫。因 View 已另外檢查 method，仍能區分成功分支；但第 15 章建議初學範例用明確 GET/POST 分支，以避免空 POST 的 bound/unbound 歧義。

19-10 寫入前再次檢查庫存

```
for item in items:
    if item.quantity > item.product.stock:
        messages.error(
            request,
            f"「{item.product.name}」庫存不足，請調整數量。",
        )
    return redirect("marketplace:cart")
```

Cart 頁看到的庫存可能在結帳前已改變，因此 checkout 必須重新驗證。

19-11 `commit=False` 補上 buyer 與 total

```
order = form.save(commit=False)
order.buyer = request.user
order.total = sum(item.subtotal for item in items)
order.save()
```

表單只提供收件欄位；View 補上 authenticated buyer 與 server-calculated total，再寫入資料庫。

19-12 建立 OrderItem 快照

```
for item in items:
    OrderItem.objects.create(
        order=order,
        product=item.product,
        seller=item.product.seller,
        product_name=item.product.name,
        unit_price=item.product.price,
        quantity=item.quantity,
    )
```

每個值都來自 database object，不從 hidden field 信任 client。

19-13 扣庫存並清 cart

```
item.product.stock -= item.quantity  
item.product.save(update_fields=["stock"])  
  
request.user.cart_items.all().delete()
```

`update_fields` 只更新 stock column。清 cart 必須在所有 order items 建立與庫存更新成功後，且仍在同一 transaction 中。

19-14 最後 redirect 到 buyer-scoped 訂單

```
messages.success(request, f"訂單 #{order.pk} 已成立。")  
return redirect("marketplace:order-detail", pk=order.pk)
```

後續 OrderDetailView 仍要用 buyer-scoped queryset。剛建立成功不代表可以讓任意登入者讀取其 URL。

19-15 `transaction.atomic` 的核心保證

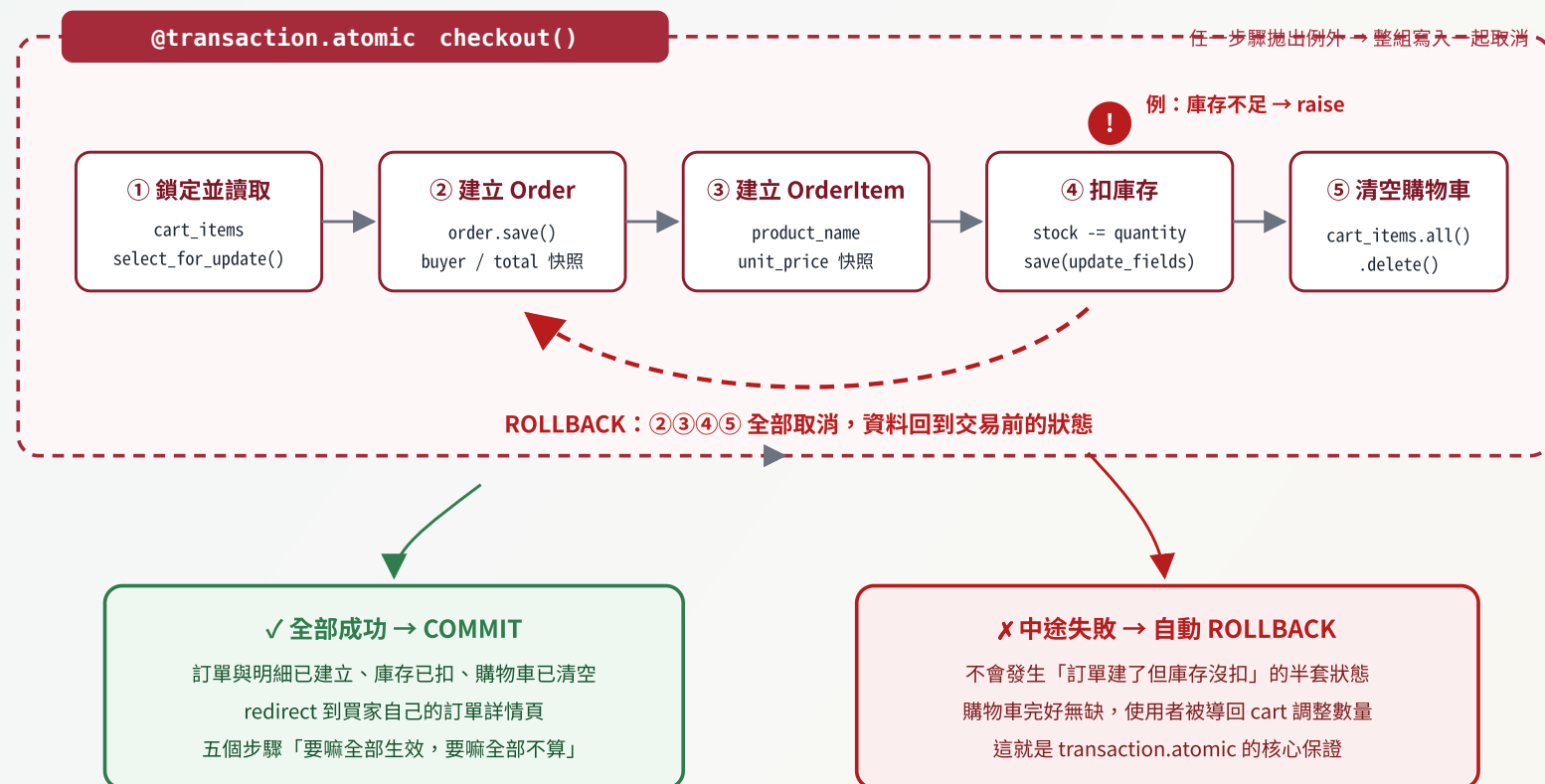
```
@transaction.atomic  
def checkout(request):  
    ...
```

在同一 database connection 上，block 內寫入：

- 全部成功 → commit
- exception 往外拋出 → rollback
- 明確標記 rollback → rollback

普通 `return` 不會自動 rollback 已完成寫入。

19-16 圖解：Checkout 的成功路徑與 rollback 路徑



19-17 為什麼 validation 要盡量在寫入前？

若先建立 Order，再發現庫存不足並普通 return，atomic 不會因「回傳錯誤頁」自動 rollback。

目前程式在任何寫入前先檢查所有 item 庫存，避免這個陷阱。

更複雜流程中可：

- raise exception
- 使用 nested atomic/savepoint
- 明確 `transaction.set_rollback(True)`

但先把 validation 排在 mutation 前最清楚。

19-18 Atomicity 與 locking 是不同問題

- **Atomicity**：一組寫入要一起 commit 或 rollback
- **Locking/concurrency control**：兩個 transaction 同時讀改同一份庫存時如何協調

只有 atomic 不一定防止 lost update；只有鎖定也不代表多筆寫入會整體 rollback。

19-19 `select_for_update()` 的意圖與範圍

```
items = request.user.cart_items.select_for_update()
```

在支援的 database 中，query evaluation 後會鎖住 query 選到的 rows，直到 transaction 結束。它必須位於 `atomic()` 中、真正 evaluation，且 target 要是共享競爭資源。

若 query 同時以 `select_related()` join 其他 tables，支援的 backend 也可能鎖 joined Product/seller rows；需要 backend 支援的 `of=(...)` 才能限制鎖定範圍。不能只看到起點是 CartItem 就斷言只鎖 CartItem。

19-20 SQLite 不提供這個 row-lock 保證

目前 LearnMart 使用 SQLite

Django 在 SQLite 上的 `select_for_update()` 沒有 row-lock effect；不會因此取得 PostgreSQL 那種資料列鎖。

所以本教學版可以理解 API 與意圖，但不能宣稱已具備 production-grade 的並行扣庫存保證。

19-21 優先明確鎖定 Product

真正競爭的共享資源是 Product stock；不同買家的 CartItem 不同。較清楚的 production 方向是：

1. 先取得 cart 中的 product PKs。
2. 在 POST 的 `atomic()` 內，以固定 PK 順序查詢 Product。
3. 對 Product queryset 使用 `select_for_update()`。
4. 用鎖後最新 stock 重新驗證與扣減。

固定順序可降低多商品 checkout 的 deadlock 風險。`of=()` 可調整 joined query 的鎖定範圍，但 explicit Product lock 更直接；仍需在支援 row locks 的 database 做 concurrency test。

19-22 Buyer 只能看自己的 Order

```
class OrderDetailView(LoginRequiredMixin, DetailView):  
    model = Order  
  
    def get_queryset(self):  
        return Order.objects.filter(  
            buyer=self.request.user  
        ).prefetch_related("items__product")
```

URL `pk` 由 generic view 在這個範圍中查找；別人的 order 會 404。

19-23 Snapshot template 不依賴目前 Product 名稱／價格

```
{% for item in order.items.all %}
  <span>{{ item.product_name }} × {{ item.quantity }}</span>
  <span>NT$ {{ item.subtotal }}</span>
{% endfor %}
```

顯示 `product_name` 與 `unit_price` 衍生的 subtotal，符合歷史快照目的。

19-24 目前 checkout test 已保護哪些行為？

```
def test_checkout_creates_order_and_reduces_stock(self):  
    ...  
    self.assertRedirects(response, order_detail_url)  
    self.assertEqual(self.product.stock, 3)  
    self.assertEqual(order.total, 1980)
```

已測：建立 order、redirect、扣庫存、server total。

尚未測：OrderItem snapshot、cart 清空、收件欄位、rollback／庫存不足。

19-25 Order privacy test 是 IDOR regression test

```
def test_user_cannot_read_another_users_order(self):  
    ...  
    response = self.client.get(  
        reverse("marketplace:order-detail", args=[order.pk])  
    )  
    self.assertEqual(response.status_code, 404)
```

它驗證 URL id 存在但不在 buyer-scoped queryset 中，View 不會洩漏他人訂單。

19-26 概念檢核

1. OrderItem 為何同時保存 FK 與 snapshot ?
2. `commit=False` 在 checkout 補了哪些 server-owned 值 ?
3. 普通 `return` 是否會讓 atomic rollback ?
4. Atomicity 與 row locking 的差異是什麼 ?
5. 為何 SQLite 上不能宣稱 `select_for_update()` 保護庫存競爭 ?

答案見配套實作手冊原第 5 章

19-27 LearnMart 實作

任務：擴充 checkout test，驗證 snapshot、cart 清空與失敗時資料不變。

驗收重點：

- OrderItem name／price／seller 正確
- 成功後 cart 為空
- 庫存不足時不建立 Order、不扣 stock、不清 cart
- 他人仍無法讀取 order

LearnMart 測試步驟與參考斷言；LearnBoard 安全／測試對應

19-28 Rollback test 要讓失敗發生在寫入之後

針對目前 checkout，可 patch `Product.save()` 在第一次庫存儲存時拋 exception。此時程式已建立 Order 與至少一個 OrderItem，才真正挑戰 view-level `@transaction.atomic`。

Exception 離開 atomic 後要檢查：

- Order 與 OrderItem 都 rollback
- database stock 不變
- CartItem 仍存在

這是同一 request/connection 的 view-level atomic rollback invariant；不是 row-lock 或 concurrent commit test。

19-29 Transaction 測試要驗證 rollback

不只測成功案例。

失敗情境應驗證：

- Order 沒建立一半
- stock 沒扣一半
- cart 沒被錯誤清空

「錯時沒有留下半套資料」才是 transaction 的價值。

第 19 章實作與離堂檢核

任務： 驗證結帳快照、庫存與購物車變化，讓中途失敗證明 rollback。

1. 展示操作結果或測試紀錄，指出對應檔案與資料。
2. 解釋一個輸入如何得到結果，以及規則在哪一層檢查。
3. 改變一個條件或製造一次失敗，記錄觀察與修正。

配套練習： [LearnMart 02 原第 5 章](#)。手冊保留原章號，對照表列出本課位置。

本份完成與後續

下一份：[14_出貨評價與狀態規則](#)。

- 保留本份操作紀錄，確認使用正確的專案與資料庫。
- 章節與實作對應可由 [全課目錄](#) 回查。